Creación del Corpus

La creación del corpus se realizó con el objetivo de obtener un conjunto de datos equilibrado y representativo de noticias reales y falsas en español para el entrenamiento y evaluación de modelos de clasificación. Este proceso se llevó a cabo mediante la integración de datasets disponibles públicamente y la recolección de datos adicionales a través de web scraping.

Fuentes de Datos y Cantidad Recopilada

Se utilizaron las siguientes fuentes para la construcción del corpus:

- **Datasets Existentes:**
  - **Spanish News Dataset (real_news_es.csv):** Este conjunto de datos contiene artículos de noticias de fuentes españolas confiables. Se aplicó un filtro para seleccionar únicamente artículos provenientes de periódicos reconocidos por su fiabilidad periodística.
  - **Noticias Falsas en Español Dataset (true_news_es.csv - onlytrue1000.csv):** A pesar de su nombre, este dataset fue utilizado para complementar el conjunto de noticias reales.
  - **Spanish Fake News Dataset (fake_news_es.csv):** Colección de noticias falsas recopiladas de diversas plataformas y formatos, de la cual se extrajo específicamente la afirmación falsa (Fake statement).
- **Web Scraping:**
  - **BBC Mundo (Sección América Latina):** Se extrajeron **aproximadamente 100 noticias** consideradas reales de esta fuente confiable.
  - **Mediterraneo Digital (Sección España Internacional):** Se recolectaron **aproximadamente 100 noticias** de este sitio web, el cual, en el contexto de este estudio, se consideró una fuente potencial de noticias falsas.

Proceso de Recopilación y Organización

El proceso de creación del corpus siguió los siguientes pasos:

- **Carga y Filtrado de Datasets:** Los datasets existentes fueron cargados y, en el caso del Spanish News Dataset, se aplicó un filtro por periódico. La columna relevante (text o Fake statement) fue seleccionada de cada dataset.

- **Recolección por Web Scraping:** Se implementaron scripts utilizando la librería Selenium para navegar por los sitios web de BBC Mundo y Mediterraneo Digital y extraer el contenido textual de los artículos. Se recolectaron aproximadamente 100 noticias de cada fuente de scraping.

- **Combinación y Muestreo:** Las noticias reales de los datasets se combinaron y se tomó una muestra aleatoria de 2500 artículos (df_sample_real_news). De manera similar, las noticias falsas de los datasets

se combinaron y se tomó una muestra aleatoria de 2500 artículos falsos (df_sample_fake_news). Las noticias obtenidas mediante web scraping se añadieron a sus respectivos conjuntos (reales o falsas).

- **Almacenamiento Local:** El texto de cada noticia real y falsa fue guardado como un archivo de texto individual (.txt) en carpetas separadas ("Verdad" y "Falso") dentro de Google Drive para facilitar su acceso y procesamiento posterior. Se guardaron un total de **2600 archivos** en la carpeta "Verdad" (2500 de datasets + ~100 de scraping) y **2600 archivos** en la carpeta "Falso" (2500 de dataset + ~100 de scraping), sumando un total de **5200 documentos** en el corpus final.

- **Creación del DataFrame Final:** Se construyó un DataFrame (df_corpus) leyendo el contenido de todos los archivos de texto almacenados. Cada fila del DataFrame representa una noticia con su texto correspondiente y una etiqueta ("1" para noticias reales y "0" para noticias falsas).

Análisis Descriptivo del Corpus

Finalmente, se realizó un análisis descriptivo básico del corpus resultante, calculando estadísticas sobre la longitud de los documentos (número de palabras) para entender la distribución del tamaño de los artículos en el conjunto de datos. Los resultados obtenidos son los siguientes:

- **Media:** 270.78 palabras
- **Desviación Estándar:** 382.17 palabras
- **Mínimo:** 0 palabras
- **Máximo:** 4558 palabras
- **Suma Total de Palabras:** 1,408,073 palabras
- **Cuartiles:** [25% (35.0), 50% (53.0), 75% (415.0)]

Estos datos indican una gran variabilidad en la longitud de los artículos dentro del corpus, con algunos textos muy cortos y otros considerablemente extensos.

Preprocesamiento, Tokenización, Reducción y Normalización

La etapa de preprocesamiento es fundamental para limpiar y transformar los datos de texto crudo, preparándolos para la extracción de características y el modelado. Este proceso implica varias sub-etapas destinadas a reducir el ruido, la dimensionalidad y unificar el formato del texto, tal como se implementa en las celdas de código correspondientes.

Limpieza del Texto

La limpieza del texto se realiza mediante la función clean_text (definida en una celda de código), que aplica una serie de transformaciones utilizando expresiones regulares (importadas con from re import compile) y la librería unicodedata para la normalización:

- **Conversión a Minúsculas:** text = text.lower()
- **Normalización y Eliminación de Acentos:** Bucle for c in unicodedata.normalize('NFD', text)...
- **Eliminación de Números:** text = RE_DIGITS.sub(' ', text)
- **Eliminación de URLs y Menciones:** text = RE_URL_MENTION.sub(' ', text)
- **Sustitución de Saltos de Línea:** text.replace("\n", " ").replace("\r", " ")
- **Eliminación de Etiquetas HTML:** text = RE_HTML_TAGS.sub(' ', text)
- **Eliminación de Signos de Puntuación:** text = RE_PUNCT.sub(' ', text)
- **Eliminación de Espacios Múltiples y Espacios en Blanco Adicionales:** text = RE_SPACES.sub(' ', text).strip()

Esta función se aplica a la columna "text" del DataFrame del corpus (df["text"].apply(clean_text)), creando una versión limpia del texto.

Tokenización

La tokenización, el proceso de dividir el texto en palabras o tokens, se realiza de forma implícita durante la etapa de vectorización al utilizar CountVectorizer y TfidfVectorizer, que dividen el texto limpio por espacios en blanco.

Reducción y Normalización (Stemming y Stopwords)

Para reducir la dimensionalidad del vocabulario y agrupar palabras con significados similares, se aplican técnicas de reducción y normalización utilizando la librería NLTK (import nltk):

- **Eliminación de Stopwords:** Se utiliza la lista de stopwords en español de NLTK (stopwords.words("spanish")). El código crea una nueva columna (df["text_stopwords"]) aplicando una función lambda que filtra las palabras que no están en la lista de stopwords.
- **Stemming:** Se utiliza el Spanish Stemmer de NLTK (SpanishStemmer()). El código crea la columna df["text_stemming"] aplicando una función lambda que reduce cada palabra a su raíz.
- **Combinación de Stopwords y Stemming:** Se crea una columna adicional (df["text_stopwords_stemming"]) aplicando el stemming al texto del corpus que ya se le habían eliminado las stopwords.

Estas transformaciones dan como resultado la creación de las columnas text_stopwords, text_stemming, y text_stopwords_stemming en el DataFrame, representando las diferentes versiones del corpus preprocesado que se utilizarán en la siguiente etapa de extracción de características.

Extracción y Ponderación de Características

Una vez que el texto ha sido preprocesado, la siguiente etapa es transformar estos datos textuales en un formato numérico que los algoritmos de machine learning puedan procesar. Esto se logra mediante la extracción y ponderación de características. En este proyecto, se

exploraron dos técnicas comunes de vectorización, implementadas utilizando la librería sklearn.feature_extraction.text.

Técnicas de Vectorización

Se utilizaron dos vectorizadores principales para convertir el texto en vectores numéricos:

- **CountVectorizer (Representación Term Frequency - TF):** Este vectorizador crea una matriz donde cada fila representa un documento y cada columna representa un término (palabra o n-grama) del vocabulario. El valor en cada celda es la frecuencia con la que el término aparece en el documento.
    - **Implementación en Código:** Se instancia como CountVectorizer(ngram_range=(1,2), min_df=3).
        - ngram_range=(1,2): Indica que se considerarán tanto unigramas (palabras individuales) como bigramas (secuencias de dos palabras consecutivas) como características.
        - min_df=3: Especifica que un término debe aparecer en al menos 3 documentos diferentes para ser incluido en el vocabulario. Esto ayuda a eliminar términos muy raros que podrían no ser informativos.
- **TfidfVectorizer (Representación TF-IDF):** Este vectorizador también crea una matriz término-documento, pero los valores en las celdas son las puntuaciones TF-IDF. La puntuación TF-IDF de un término en un documento es proporcional a cuántas veces aparece el término en el documento (TF) e inversamente proporcional a la frecuencia con la que aparece el término en todo el corpus (IDF). Esto da más peso a los términos que son relevantes para un documento específico, pero no son demasiado comunes en el corpus general.
    - **Implementación en Código:** Se instancia como TfidfVectorizer(ngram_range=(1,2), min_df=3).
        - ngram_range=(1,2): Similar a CountVectorizer, considera unigramas y bigramas.
        - min_df=3: Similar a CountVectorizer, requiere que un término aparezca en al menos 3 documentos.

Aplicación a los diferentes Datasets Preprocesados

Estos vectorizadores se aplicaron a cada una de las cuatro versiones del corpus preprocesado (original limpio, con stopwords, con stemming, y con stopwords + stemming). Esto se gestiona en el código mediante bucles que iteran sobre los diccionarios vectorizers_feat y datasets.

Para cada combinación de dataset preprocesado y vectorizador, se realiza lo siguiente:

- **Ajuste y Transformación:** El vectorizador se ajusta a los datos del corpus preprocesado (vectorizer.fit_transform(corpus)). Este paso aprende el vocabulario del corpus y transforma el texto en la matriz de características numéricas (X).

- **Definición de la Variable Objetivo:** La variable objetivo (y) se define como la columna "type" del DataFrame principal (df["type"]), que indica si la noticia es real ("1") o falsa ("0").

El resultado de esta etapa son las matrices de características (X) para cada combinación de técnica de preprocesamiento y vectorización, listas para ser utilizadas en el entrenamiento de los modelos de machine learning. La dimensionalidad exacta de estas matrices (número de características) dependerá del vocabulario resultante después de aplicar min_df y considerar los n-gramas para cada dataset preprocesado

Técnicas de aprendizaje

Las técnicas de aprendizaje utilizadas fueron Regresión logística, árboles de decisión, K vecinos más cercanos y máquina de vectores de soporte. Para cada una de ellas, se implementó un esquema de validación cruzada con 10 iteraciones como se indicaba en la práctica, con el fin de evitar sobreajuste y reducir la dependencia de una partición particular de los datos que tal vez podría sesgar los resultados, asegurando así una evaluación más estable y representativa de los modelos. Se incluyeron las métricas de rendimiento balanceadas (accuracy, precisión, recall y F1-score) como se necesitaban para el análisis, y se destinó el 80% del corpus para entrenamiento y el 20% para test, tal como se pedía, para reservar una parte de los datos que sirvan para probar los modelos con noticias no vistas antes.

El modelo de regresión logística suele ser el más clásico en casos de clasificación binaria como este, por lo que su implementación representa, a priori, una base sólida para distinguir las noticias falsas y verdaderas. Se configuró con un máximo de 2000 iteraciones, valor que suele ser más que suficiente y genera confianza para la convergencia del optimizador.

El modelo de Árbol de Decisión permite hacer clasificaciones sucesivas basadas en distintas condiciones y características, lo que es útil para este caso dada la gran variedad de las noticias. Se establecieron parámetros como una profundidad máxima de 15 y un mínimo de muestras por división y hoja de 50, ya que para la cantidad considerable de datos, estos valores deberían ser suficientes para la clasificación sin llegar a un árbol demasiado específico que genere sobreajuste.

El modelo de K vecinos más cercanos clasifica según las etiquetas de los datos más cercanos, en este caso comparando los mensajes vectorizados del corpus. Se definió un valor de 15 vecinos a comparar, pues un valor muy pequeño capta mucho ruido y tampoco conviene aumentar mucho este valor debido a la diversidad de noticias y las distintas representaciones que pueden tener, lo que puede provocar que se relacionen vecinos inadecuados. Se usó también la métrica de similitud coseno que es ideal para comparar representaciones de texto basadas en la frecuencia de términos y captar enfoques en lugar de longitudes, junto con el algoritmo de búsqueda exhaustiva que es el más compatible con esta métrica. Por último, se dió más peso a los vecinos más cercanos porque, como se mencionó anteriormente, hay mucha variedad en los datos y esto evita ambigüedades.

Y el modelo de máquina de vectores de soporte se implementó con los parámetros clásicos al tratarse de por sí de un modelo especialista en clasificación binaria. Su comportamiento por defecto destaca un kernel radial que identifica fronteras de separación no lineales, como es de esperarse en nuestro corpus variado, y maneja muy bien la alta dimensionalidad que presentan los vectores de texto que usamos.

Finalmente, las matrices de características mencionadas en el punto anterior, fueron evaluadas en cada uno de estos 4 modelos. Todo esto con el fin de obtener una tabla comparativa nutrida con los resultados de cada configuración posible, para así poder analizar mejor el rendimiento de los modelos según las condiciones de cada entrada. Los resultados de la tabla se presentan a continuación:

	REGRESIÓN LOGÍSTICA					
#	**POND**	**STOPWORDS**	**STEMMING**	**PRECISIÓN**	**RECALL**	**F1-SCORE**
1	TO	FALSE	FALSE	0.849563	0.846394	0.845994
2	TO	TRUE	FALSE	0.851229	0.847115	0.846577
3	TO	FALSE	TRUE	0.857030	0.854327	0.854003
4	TO	TRUE	TRUE	0.854604	0.851683	0.851326
5	TF-IDF	FALSE	FALSE	0.817247	0.812981	0.812242
6	TF-IDF	TRUE	FALSE	0.851550	0.845673	0.844944
7	TF-IDF	FALSE	TRUE	0.825591	0.821875	0.821277
8	TF-IDF	TRUE	TRUE	0.849756	0.844952	0.844342

	MÁQUINA DE VECTORES DE SOPORTE					
#	**POND**	**STOPWORDS**	**STEMMING**	**PRECISIÓN**	**RECALL**	**F1-SCORE**
1	TO	FALSE	FALSE	0.791575	0.751202	0.742126
2	TO	TRUE	FALSE	0.800197	0.761538	0.753533
3	TO	FALSE	TRUE	0.797883	0.756490	0.747591
4	TO	TRUE	TRUE	0.802081	0.762740	0.754705

5	TF-IDF	FALSE	FALSE	0.842151	0.834375	0.833340
6	TF-IDF	TRUE	FALSE	0.864646	0.856250	0.855363
7	TF-IDF	FALSE	TRUE	0.852173	0.845192	0.844352
8	TF-IDF	TRUE	TRUE	0.863354	0.856010	0.855239

	ÁRBOLES DE DECISIÓN					
#	**POND**	**STOPWORDS**	**STEMMING**	**PRECISIÓN**	**RECALL**	**F1-SCORE**
1	TO	FALSE	FALSE	0.772226	0.770192	0.769643
2	TO	TRUE	FALSE	0.730328	0.725721	0.724114
3	TO	FALSE	TRUE	0.772047	0.770192	0.769703
4	TO	TRUE	TRUE	0.740665	0.737260	0.736175
5	TF-IDF	FALSE	FALSE	0.749821	0.748798	0.748471
6	TF-IDF	TRUE	FALSE	0.722683	0.719712	0.718581
7	TF-IDF	FALSE	TRUE	0.751003	0.749760	0.749416
8	TF-IDF	TRUE	TRUE	0.733201	0.731010	0.730235

	K VECINOS MÁS CERCANOS					
#	**POND**	**STOPWORDS**	**STEMMING**	**PRECISIÓN**	**RECALL**	**F1-SCORE**
1	TO	FALSE	FALSE	0.746950	0.560817	0.458500
2	TO	TRUE	FALSE	0.818015	0.815625	0.815290
3	TO	FALSE	TRUE	0.752485	0.563221	0.462370
4	TO	TRUE	TRUE	0.813559	0.809135	0.808453
5	TF-IDF	FALSE	FALSE	0.784252	0.697837	0.672892
6	TF-IDF	TRUE	FALSE	0.844674	0.844231	0.844182
7	TF-IDF	FALSE	TRUE	0.787622	0.693750	0.666370
8	TF-IDF	TRUE	TRUE	0.845259	0.844471	0.844385

Conclusiones

Las conclusiones de este trabajo se construyen a partir de la tabla de resultados y de las gráficas comparativas generadas en el notebook adjunto. Tal como se indicaba en la práctica, la métrica de referencia utilizada principalmente fue el F1 ponderado, esto debido a que el F1 integra en un solo indicador tanto la precisión como el recall, lo cual resulta fundamental en problemas como este de detección de noticias falsas, donde tanto los falsos positivos como los falsos negativos tienen consecuencias importantes. Además, se usó la versión ponderada, lo que permite que la evaluación tenga presente el peso real de cada clase, evitando el hipotético caso donde pudiera existir un desbalance entre noticias verdaderas y falsas.

Al observar el F1 máximo alcanzado por algoritmo, se encuentra que tanto la Regresión Logística como el SVM alcanzaron los mejores valores, seguidos de KNN y el Árbol de Decisión. Esto muestra que, en términos generales, los algoritmos matemáticos de optimización se desempeñan mejor en la clasificación de noticias por su esencia vectorial, mientras que los otros métodos basados más en reglas simples y decisiones locales muestran un rendimiento un poco más limitado.

En el análisis del F1 medio según tipo de representación vectorial (TO vs TF-IDF) se observa que la Regresión Logística obtiene mejor desempeño con TO, lo que indica que en este caso la frecuencia absoluta de los términos aporta más información que la ponderada. El SVC, en cambio, muestra un comportamiento inverso: con TF-IDF obtiene un mejor rendimiento que con TO, lo cual sugiere que este modelo aprovecha mejor la ponderación de términos distintivos. En KNN también se aprecia una ventaja clara de TF-IDF, coherente con el hecho de que este algoritmo depende directamente de la medida de similitud entre documentos, y la normalización de TF-IDF ayuda a medir mejor esa cercanía. Por el

contrario, en Árbol de Decisión el resultado es más estable, con ambas técnicas el F1 no presenta grandes variaciones.

Respecto al F1 medio por técnica de reducción aplicada al corpus, se encuentra que en general los modelos matemáticos (LR y SVC) presentan mejoras modestas cuando se aplican reducciones, alcanzando sus mejores valores con la combinación de stopwords y stemming. Esto sugiere que la limpieza adicional ayuda a eliminar ruido, pero no transforma radicalmente el rendimiento en estos casos. En cambio, en KNN se aprecia un cambio mucho más fuerte: con texto original o solo stemming el F1 se queda muy corto, mientras que con stopwords o con ambas reducciones sube significativamente, lo que confirma que este algoritmo depende fuertemente de un preprocesamiento cuidadoso que evite la inclusión de palabras ruidosas para que no se calculen en las distancias y funcione adecuadamente con los términos representativos. Finalmente, en Árbol de Decisión las reducciones no muestran impacto significativo, manteniendo valores similares en todas las variantes como en el punto anterior.

En conjunto, los resultados permiten concluir que la Regresión Logística (ligeramente favorita) y el SVC son los modelos más eficaces para el problema de clasificación de noticias planteado, principalmente si están acompañados de reducción con stopwords como mínimo. La diferencia es que Regresión logística funciona mejor con TO y SVC con TF-IDF, ya que en la primera es muy valorada la magnitud y en la segunda el peso de la aparición. Para esta implementación, ambos modelos alcanzaron F1-score de hasta un 86%, lo que puede considerarse bueno para el tamaño tan considerable del corpus. Por otra parte, el modelo KNN presenta un buen rendimiento bajo algunas configuraciones específicas (como TF-IDF con stopwords) pero aún así no alcanza a ser igual de eficaz que los otros dos, mientras que el árbol de decisión es muy poco sensible a las configuraciones porque se basa en seguir sus propias reglas, lo que sin duda lo hace el modelo menos adecuado para la clasificación de texto.